

Angular Unit Testing with Vitest

Vitest framework

Vitest is the default testing framework for Angular 21+ and is used to run unit tests for components, services, pipes, directives, and other application logic.

Angular integrates Vitest directly through the CLI, providing support for global setup files and shared providers.

For applications with common dependencies such as routing, using `providersFile` can significantly reduce test boilerplate and keep test code clean.

Angular starts the Vitest test runner, discovers all test files (*.spec.ts and *.test.ts), compiles the Angular application, and executes the tests. By default, tests run in a Node.js environment using a simulated browser DOM provided by jsdom. This allows tests to run quickly without launching a real browser.

The process is:

1. Angular builds the test environment.
2. Vitest discovers test files.
3. Angular's TestBed creates components and injects services.
4. Vitest executes assertions.
5. Results are displayed in the terminal

Spec Files are also standalone.

Each test file must import any dependencies that is required to do its test. As each component is tested independent of rest of the application. Some of the config used for the app that was setup in the `app.config.ts` file also need to be invoked for the test.

Global setups in angular.json file

Global Setup Files

You can use the `setupFiles` option in `angular.json` to execute code before tests run, such as configuring mocks or polyfills.

Global Providers

Angular also supports a `providersFile` option. This allows providers to be registered once for every test instead of repeatedly adding them to individual specs.

```
"test": {  
  "builder": "@angular/build:unit-test",  
  "options": {  
    "providersFile": "src/test-providers.ts"  
  }  
}
```

The test-providers might typically look like.

```
1  import { provideRouter } from '@angular/router';
2  import { provideHttpClient } from '@angular/common/http';
3  import { provideHttpClientTesting } from '@angular/common/http/testing'
4
5
6  export default [
7    provideRouter([]),
8    provideHttpClient(),
9    provideHttpClientTesting(),
10 ];
11
```

Using the global providers means you do not need to add these to each test file individually.

Older Options

A Vitest test in Angular follows the same overall pattern as the older Jasmine/Karma tests, using Angular's TestBed to create an isolated Angular environment and Vitest's functions (describe, it, expect, beforeEach) to define and execute tests.

Angular's built-in testing support integrates TestBed with Vitest, allowing components, services, directives, and pipes to be tested in a realistic Angular environment.

What makes up a test

A typical Angular + Vitest test consists of:

Imports – component/service and testing utilities.

describe() – groups related tests.

beforeEach() – configures TestBed.

Fixture creation – creates the component under test.

detectChanges() – triggers Angular rendering.

it() blocks – individual test cases.

expect() assertions – verify behaviour.

vi mocks/spies – replace or monitor dependencies.

Fixtures

A *ComponentFixture* $\langle T \rangle$ is a wrapper around an Angular component instance and its rendered template.

When you create a component in a test:

```
TypeScriptFixture = TestBed.createComponent(MyComponent);
```

Angular doesn't just give you the component instance. It creates a fixture that contains:

- The component instance
- The rendered DOM
- Change detection capabilities
- Access to Angular debugging utilities

Think of the fixture as a test harness that lets you interact with both the TypeScript class and the HTML template

Each test/spec within the suite is defined by calling the function **it()**.

```
describe('add two numbers together',() =>{  
  it("should be able to add two whole numbers",()=>{  
    expect(adding.add(2,2)).toEqual(4);  
  });  
  it("should be able to add positive and negative numbers",()=>{  
    expect(adding.add(2,-1)).toEqual(1);  
  });  
});
```

Within each **spec** we call **expect** and call the actual code. A **matcher** function (toEqual()) is then called to test the expected outcome. There are many matchers available to test a range of conditions.

Mocking

Mocking is a process of creating a known dataset and passing this to our test rather than hitting an actual service for live data. This lets us isolate out testing variables to just the component , class or function being tested. The external data source is not being tested as part of unit testing. That can come later in E2E testing.

The mocked data can then easliy contain all of the variety of test cases that might be harder to generate from a live service.

This does rely on your mocked data matching the expected output of a service.

Structure of Test File

- Import all the angular testing tools and depending components
- Configure an angular test bed in "beforeEach" with decarations, imports and providers. After setting the configuration, you call the compile components function.
- A Global provider has been used to provides access to routing.

```
1  import { TestBed } from '@angular/core/testing';
2  import { App } from './app';
3
4
5  describe('App', () => {
6    beforeEach(async () => {
7      await TestBed.configureTestingModule({
8        imports: [App]
9      }).compileComponents();
10   });
11
12   it('should create the app', () => {
13     const fixture = TestBed.createComponent(App);
14     const app = fixture.componentInstance;
15     expect(app).toBeTruthy();
16   });
17
18 });
19
```

Component Instance and Native Element

- In the first test, we are checking that the component actually contains the expected text in the “title” property. With the instance of app.component (fixture) we can check the value in the title property.
- The second test checks that the dom renders the “title” property. It does the same as the first test, gets the app.component fixture. After the detectchanges() function, it gets the native element of the compiled HTML (the HTML rendered by the component). With the native element, we select the “h1” containing the “title” value and expect that the selected HTML contains the expected value.

```
it('should have as title 'Angular Unit Testing'', async(() => {
  const fixture = TestBed.createComponent(AppComponent);
  const app = fixture.debugElement.componentInstance;
  expect(app.title).toEqual('Angular Unit Testing');
}));

it('should render title in a h1 tag', async(() => {
  const fixture = TestBed.createComponent(AppComponent);
  fixture.detectChanges();
  const compiled = fixture.debugElement.nativeElement;
  expect(compiled.querySelector('h1').textContent).toContain('Welcome to Angular Unit Testing!');
}));
```

Testing a Form

In this example, the imports section, introduces the “By” that is going to allow us to select elements from the DOM.

In the beforeEach, the promise is used for the “compileComponents” function returns. When the promise is resolved we are given a value to each of the variables that we declare at the beginning.

```
import { TestBed, async, ComponentFixture } from '@angular/core/testing';
import { BrowserModule, By } from '@angular/platform-browser';
import { FormsModule, ReactiveFormsModule } from '@angular/forms';
import { DebugElement } from '@angular/core';

import { ContactComponent } from './contact.component';

describe('ContactComponent', () => {
  let comp: ContactComponent;
  let fixture: ComponentFixture<ContactComponent>;
  let de: DebugElement;
  let el: HTMLElement;

  beforeEach(async(() => {
    TestBed.configureTestingModule({
      declarations: [
        ContactComponent
      ],
      imports: [
        BrowserModule,
        FormsModule,
        ReactiveFormsModule
      ]
    }).compileComponents().then(() => {
      fixture = TestBed.createComponent(ContactComponent);

      comp = fixture.componentInstance; // ContactComponent test instance

      de = fixture.debugElement.query(By.css('form'));
      el = de.nativeElement;
    });
  }));
});
```

Testing a Form

Let us take the second and third tests as example to explain.

- In the second test the property of the component “submitted” is set to true when the “onSubmit” function is called.
- The third test first detects the component state to the HTML with the function “detectChanges” of the “fixture” object and creates a jasmine “spy” on the “onSubmit” function of the component. Then it gets the submit button from the DOM and triggers the click event. It expects that the spied function is not executed, because the button should be disabled since the form is not valid.

```
it('should have as text 'contact page'', async(() => {
  expect(comp.text).toEqual('contact page');
}));

it('should set submitted to true', async(() => {
  comp.onSubmit();
  expect(comp.submitted).toBeTruthy();
}));

it('should call the onSubmit method', async(() => {
  fixture.detectChanges();
  spyOn(comp, 'onSubmit');
  el = fixture.debugElement.query(By.css('button')).nativeElement;
  el.click();
  expect(comp.onSubmit).toHaveBeenCalledTimes(0);
}));

it('form should be invalid', async(() => {
  comp.contactForm.controls['email'].setValue('');
  comp.contactForm.controls['name'].setValue('');
  comp.contactForm.controls['text'].setValue('');
  expect(comp.contactForm.valid).toBeFalsy();
}));
```

Testing a Service

- `TestBed.configureTestingModule` sets up the service we want to test.
- The service is injected into our test suite using `TestBed.inject()` with the service we want to test as the argument.
- The local service variable allows us to interact with this service within our tests.

```
1  import { TestBed } from '@angular/core/testing';
2
3  import { Auth } from '../auth';
4
5  describe('Auth', () => {
6    let service: Auth;
7
8    beforeEach(() => {
9      TestBed.configureTestingModule({});
10     service = TestBed.inject(Auth);
11   });
12
13   it('should be created', () => {
14     expect(service).toBeTruthy();
15   });
16 });
```

Testing a Component with Services

- Suppose we have a simple component that gets the list of users from a service.
- Let's see how we create the tests for the component mocking the service implementation.

```
import { UserService } from './user.service';

@Component({
  templateUrl: './user.component.html',
  styleUrls: ['./user.component.sass']
})
export class UserComponent {
  text = 'user page';
  users;

  constructor(private userService: UserService) {
    this.users = this.userService.getUsers();
  }
}
```

Testing a Component with Services

we don't actually hit the API when we're testing various scenarios. We mock the response we may expect to receive. That is a difference between unit test and e2e tests for angular

In the provider's declaration of the test module, the service "UserService" is injected and uses the "UserServiceMock" class.

The "UserServiceMock" is a dummy service that is created to return dummy data to run the tests of the component

```
import { TestBed, async, ComponentFixture } from '@angular/core/testing';
import { BrowserModule, By } from '@angular/platform-browser';
import { DebugElement } from '@angular/core';

import { UserComponent } from '../user.component';
import { UserServiceMock } from '../../mocks/user.service.mock';

describe('ContactComponent', () => {
  let comp: UserComponent;
  let fixture: ComponentFixture<UserComponent>;

  beforeEach(async(() => {
    TestBed.configureTestingModule({
      declarations: [
        UserComponent
      ],
      providers: [
        { provide: UserService, useClass: UserServiceMock }
      ]
    }).compileComponents().then(() => {
      fixture = TestBed.createComponent(UserComponent);
      comp = fixture.componentInstance; // UserComponent test instance
    });
  }));
});
```

Testing a Component with Services

Now we get

- a test instance **comp** for UserComponent and
- the native element **el** for the HTML rendered by the component

The two test can be carried out easily.

```
it(`should have one user`, async(() => {  
  expect(comp.users.length).toEqual(1);  
}));  
  
it(`html should render one user`, async(() => {  
  fixture.detectChanges();  
  const el = fixture.nativeElement.querySelector('p');  
  expect(el.innerText).toContain('user1');  
}));
```

More information on Testing

More information on unit testing can be found at:

<https://angular.dev/guide/testing>

A good video series to watch on YouTube

<https://www.youtube.com/watch?v=HCTG1XKdkG8&list=PLCKuOXG0bPi2T9u1GeWrXQmrAcEEamMGf>